LAPORAN PRAKTIKUM SISTEM BASIS DATA

BAB 4 DAN 5 DATA QUERY SELECT DAN

DATA MANIPULATION LANGUAGE (DML)

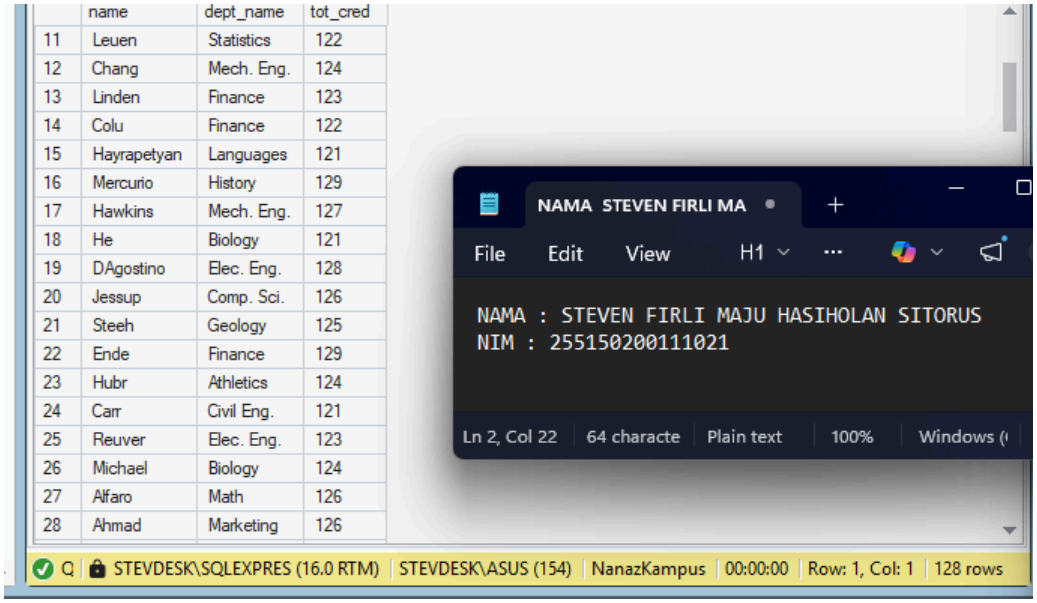
Nama : STEVEN FIRLI MAJU HASIHOLAN SITORUS
Program Studi : TEKNIK INFORMATIKA
Kelas : BASIS DATA B
Asisten 1 : AFFAN ABYARAHMAN
Asisten 2 : MUHAMMAD FAKHRI ALDIANSYAH

DATA QUERY SELECT

1. **Implementasi**

```
1  SELECT name, dept_name, tot_cred
2  FROM student
3  WHERE tot_cred > 120;
```

Tampilan Keluaran



	name	dept_name	tot_cred
11	Leuen	Statistics	122
12	Chang	Mech. Eng.	124
13	Linden	Finance	123
14	Colu	Finance	122
15	Hayrapetyan	Languages	121
16	Mercurio	History	129
17	Hawkins	Mech. Eng.	127
18	He	Biology	121
19	DAgostino	Elec. Eng.	128
20	Jessup	Comp. Sci.	126
21	Steeh	Geology	125
22	Ende	Finance	129
23	Hubr	Athletics	124
24	Carr	Civil Eng.	121
25	Reuver	Elec. Eng.	123
26	Michael	Biology	124
27	Alfaro	Math	126
28	Ahmad	Marketing	126

Pembahasan dan Analisis

Kode implementasi digunakan untuk mengambil data nama, nama program studi, dan total SKS mahasiswa. Data diambil dari tabel student tanpa melalui operasi join. Baris ke-1 digunakan untuk menentukan kolom spesifik yang akan diproyeksikan, yaitu name, dept_name, dan tot_cred. Baris ke-2 menunjukkan tabel sumber data pembacaan, yaitu student. Baris ke-3 menunjukkan kondisi seleksi untuk memfilter data, memastikan hanya mahasiswa dengan SKS di atas 120 yang diproses.

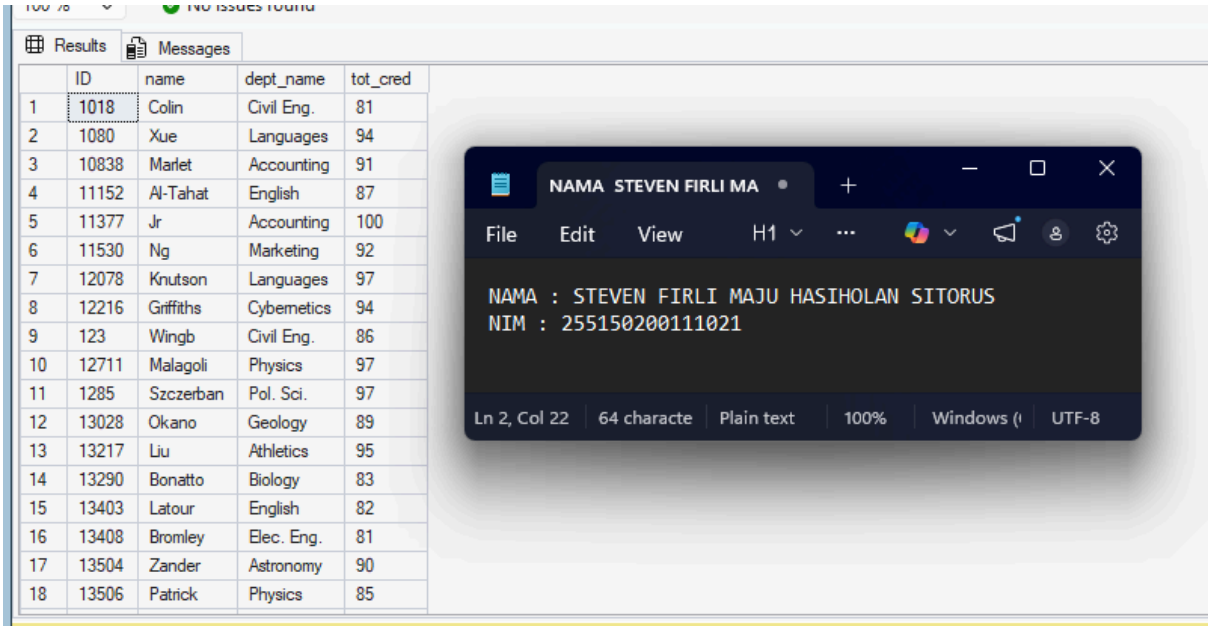
Dari percobaan dan analisis yang telah dilakukan, apabila baris 3 tidak ada maka hasilnya seluruh data mahasiswa di dalam tabel akan ditampilkan tanpa terkecuali (termasuk yang SKS-nya di bawah atau sama dengan 120). Sedangkan apabila baris 1 diubah menjadi SELECT *,

maka hasilnya seluruh kolom pada tabel student akan ikut ditampilkan, sehingga supaya dapat mengambil data spesifik sesuai permintaan harus menuliskan nama kolom secara persis pada klausa SELECT dengan mempertimbangkan struktur tabel asal. Dilanjutkan dengan penjelasan lain, ini adalah bentuk paling dasar dari operasi proyeksi (memilih kolom) dan seleksi (memilih baris) dalam basis data.

2. Implementasi

```
1  SELECT *
2  FROM student
3  WHERE tot_cred > 80 AND tot_cred <= 100;
```

Tampilan Keluaran



Results | Messages

	ID	name	dept_name	tot_cred
1	1018	Colin	Civil Eng.	81
2	1080	Xue	Languages	94
3	10838	Marlet	Accounting	91
4	11152	Al-Tahat	English	87
5	11377	Jr	Accounting	100
6	11530	Ng	Marketing	92
7	12078	Knutson	Languages	97
8	12216	Griffiths	Cybernetics	94
9	123	Wingb	Civil Eng.	86
10	12711	Malagoli	Physics	97
11	1285	Szczerban	Pol. Sci.	97
12	13028	Okano	Geology	89
13	13217	Liu	Athletics	95
14	13290	Bonatto	Biology	83
15	13403	Latour	English	82
16	13408	Bromley	Elec. Eng.	81
17	13504	Zander	Astronomy	90
18	13506	Patrick	Physics	85

Pembahasan dan Analisis

Kode implementasi digunakan untuk mengambil seluruh kolom data mahasiswa yang rentang SKS-nya berada di atas 80 hingga maksimal 100. Data diambil dari tabel student tanpa operasi join. Baris ke-1 digunakan untuk mengambil seluruh kolom yang tersedia di tabel menggunakan simbol asterisk (*). Baris ke-2 menunjukkan tabel yang menjadi sumber data. Baris ke-3 menunjukkan penggunaan operator relasional komposit dengan AND, di mana data harus memenuhi dua syarat sekaligus: lebih dari 80 (> 80) dan kurang dari atau sama dengan 100 (<= 100).

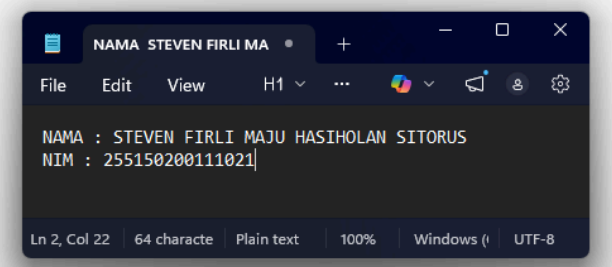
Dari percobaan dan analisis yang telah dilakukan, apabila baris 3 tidak ada maka hasilnya semua baris mahasiswa akan muncul tanpa terfilter oleh batasan rentang nilai SKS. Sedangkan apabila kata AND diganti menjadi OR pada baris 3, maka hasilnya data yang ditampilkan akan salah karena semua SKS akan dianggap valid (memenuhi salah satu dari >80 atau <=100), sehingga supaya dapat mengambil data pada rentang interval tertentu yang ketat, harus menggunakan operator logika AND dengan mempertimbangkan batasan bawah dan batas atas nilai menggunakan operator >, <, >=, atau <=.

3. Implementasi

```
1  SELECT *
2  FROM student
3  WHERE tot_cred BETWEEN 81 AND 100;
```

Tampilan Keluaran

	ID	name	dept_name	tot_cred
136	46769	Kivv	Psychology	85
137	47824	Hardt	Astronomy	94
138	48471	Elias	Languages	90
139	4860	Penneb	Elec. Eng.	83
140	49205	Agraz	Pol. Sci.	82
141	49280	Wilson	Psychology	89
142	49450	Gotoh	Astronomy	83
143	49813	Al-Hu	History	93
144	49873	Nikut	Athletics	85
145	50743	Lindner	Biology	83
146	50966	Concilio	Finance	95
147	50977	Allen	Elec. Eng.	93
148	51093	Syng	Psychology	92
149	51203	Lahtinen	Math	99
150	51549	Rao	Mech. Eng.	87
151	51579	Holz	Biology	89
152	51698	Sharpe	History	81
153	51723	Lagendijk	Comp. Sci.	99
154	52076	Noda	Astronomy	85
155	52187	Fritsch	Math	98
156	5243	Bajracha...	Marketing	83
157	53077	Brandt	History	87
158	53089	Vedag	English	98
159	53165	Dowey	History	90
160	53225	Juan	Finance	92
161	53451	Hasan	Physics	83
162	53588	Schwet	Languages	100
163	53803	Okaf	Civil Eng.	89
164	54296	Osinski	Elec. Eng.	91
165	544	Uchiyama	Biology	81
166	54460	Martini	Math	100
167	5463	Reiss	Finance	95
168	54875	Feng	Statistics	83
169	55531	Apostolov	Physics	85
170	56	Kameda	Cybernetics	81



Pembahasan dan Analisis

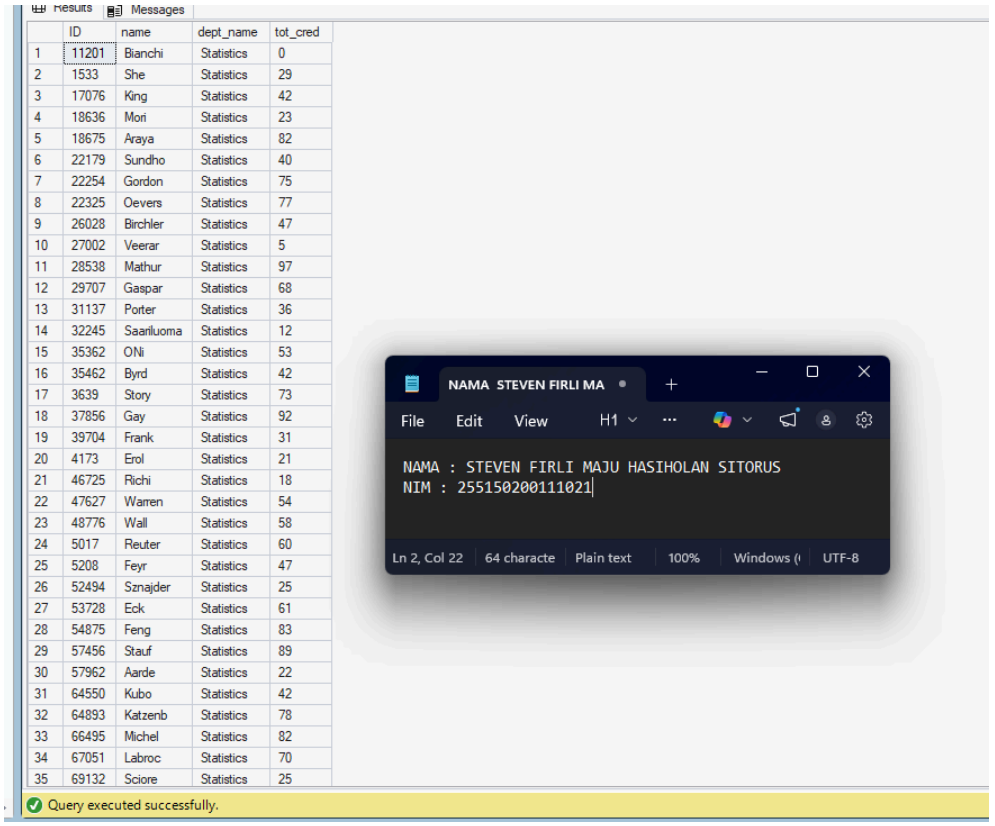
Kode implementasi digunakan untuk mengambil seluruh data mahasiswa dengan total SKS pada rentang lebih dari 80 hingga 100 menggunakan fungsi bawaan SQL. Data diambil dari tabel student. Baris ke-1 digunakan untuk memanggil seluruh kolom pada tabel. Baris ke-2 menunjukkan tabel referensi utama. Baris ke-3 menunjukkan klausa WHERE yang diiringi operator BETWEEN, yang berfungsi menyeleksi rentang nilai inklusif (termasuk angka batasnya). Karena instruksinya adalah "lebih dari 80", maka nilai awal yang dipakai pada klausa BETWEEN adalah 81.

Dari percobaan dan analisis yang telah dilakukan, apabila klausa BETWEEN pada baris 3 ditulis sebagai BETWEEN 80 AND 100, maka hasilnya mahasiswa dengan SKS tepat 80 akan ikut terambil padahal instruksinya meminta nilai lebih dari 80. Sedangkan apabila baris 3 tidak ada maka hasilnya sama seperti query tanpa filter lainnya (menampilkan semua data), sehingga supaya dapat mengambil data rentang menggunakan BETWEEN harus memasukkan nilai batas bawah secara akurat dengan mempertimbangkan sifat inklusif dari perintah BETWEEN itu sendiri.

4. Implementasi

```
1  SELECT *
2  FROM student
3  WHERE dept_name = ‘Statistics’ AND tot_cred <= 100;
```

Tampilan Keluaran



Results | Messages

	ID	name	dept_name	tot_cred
1	11201	Bianchi	Statistics	0
2	1533	She	Statistics	29
3	17076	King	Statistics	42
4	18636	Mori	Statistics	23
5	18675	Araya	Statistics	82
6	22179	Sundho	Statistics	40
7	22254	Gordon	Statistics	75
8	22325	Oevers	Statistics	77
9	26028	Birchler	Statistics	47
10	27002	Veerar	Statistics	5
11	28538	Mathur	Statistics	97
12	29707	Gaspar	Statistics	68
13	31137	Porter	Statistics	36
14	32245	Saariluoma	Statistics	12
15	35362	ONi	Statistics	53
16	35462	Byrd	Statistics	42
17	3639	Story	Statistics	73
18	37856	Gay	Statistics	92
19	39704	Frank	Statistics	31
20	4173	Erol	Statistics	21
21	46725	Richi	Statistics	18
22	47627	Warren	Statistics	54
23	48776	Wall	Statistics	58
24	5017	Reuter	Statistics	60
25	5208	Feyr	Statistics	47
26	52494	Sznajder	Statistics	25
27	53728	Eck	Statistics	61
28	54875	Feng	Statistics	83
29	57456	Stauf	Statistics	89
30	57962	Aarde	Statistics	22
31	64550	Kubo	Statistics	42
32	64893	Katzenb	Statistics	78
33	66495	Michel	Statistics	82
34	67051	Labroc	Statistics	70
35	69132	Sciore	Statistics	25

Query executed successfully.

Pembahasan dan Analisis

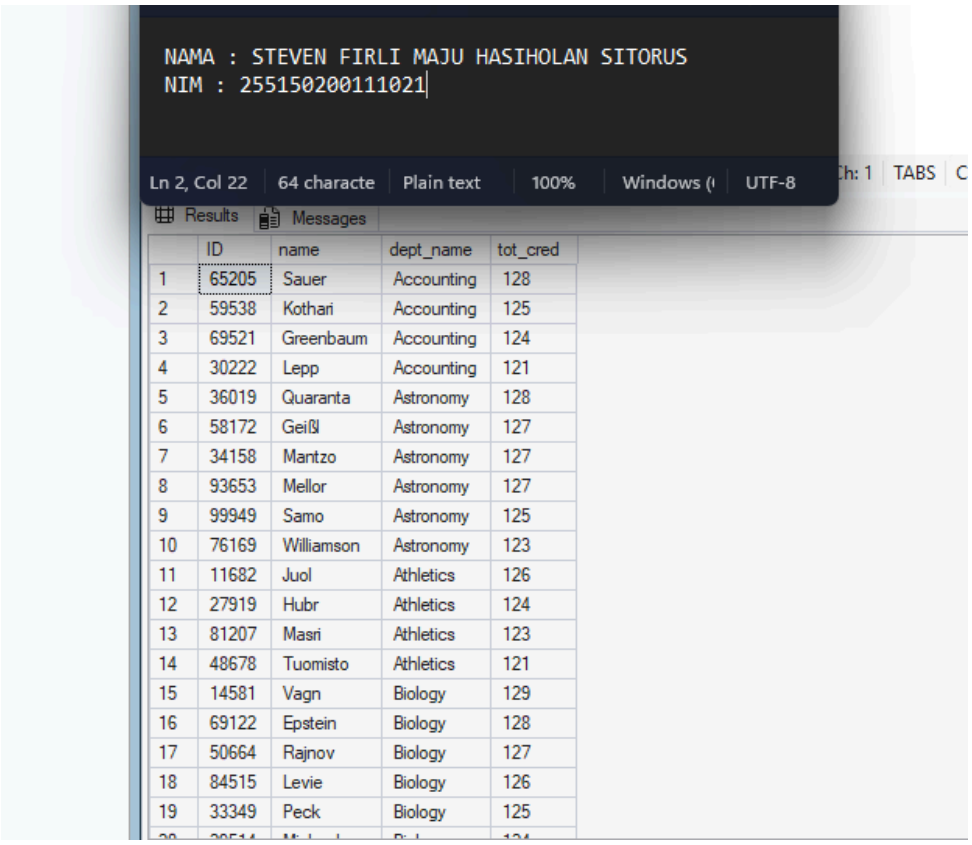
Kode implementasi digunakan untuk mengambil data mahasiswa spesifik dari departemen Statistika yang mengambil SKS maksimal 100. Data diambil dari tabel student. Baris ke-1 digunakan untuk menampilkan semua kolom data. Baris ke-2 menunjukkan pengaksesan tabel student. Baris ke-3 menunjukkan penerapan dua kondisi seleksi yang digabung dengan AND, yaitu memastikan nama departemen bertuliskan eksak 'Statistics' dan tot_cred bernilai kurang dari atau sama dengan 100.

Dari percobaan dan analisis yang telah dilakukan, apabila baris 3 hanya menyertakan dept_name = 'Statistics' tanpa kondisi SKS, maka hasilnya mahasiswa Statistika dengan SKS lebih dari 100 juga akan ikut ditampilkan. Sedangkan apabila penulisan 'Statistics' pada baris 3 tidak menggunakan tanda kutip tunggal ('...'), maka hasilnya program akan mengalami syntax error, sehingga supaya dapat mengambil data bertipe string/varchar harus selalu diapit tanda kutip tunggal dengan mempertimbangkan bahwa case-sensitivity (huruf besar/kecil) sering kali berpengaruh pada isi teks di dalam database.

5. **Implementasi**

```
1  SELECT *
2  FROM student
3  WHERE tot_cred > 120
4  ORDER BY dept_name ASC, tot_cred DESC, name ASC;
```

Tampilan Keluaran



	ID	name	dept_name	tot_cred
1	65205	Sauer	Accounting	128
2	59538	Kothari	Accounting	125
3	69521	Greenbaum	Accounting	124
4	30222	Lepp	Accounting	121
5	36019	Quaranta	Astronomy	128
6	58172	Geißl	Astronomy	127
7	34158	Mantzo	Astronomy	127
8	93653	Mellor	Astronomy	127
9	99949	Samo	Astronomy	125
10	76169	Williamson	Astronomy	123
11	11682	Juol	Athletics	126
12	27919	Hubr	Athletics	124
13	81207	Masri	Athletics	123
14	48678	Tuomisto	Athletics	121
15	14581	Vagn	Biology	129
16	69122	Epstein	Biology	128
17	50664	Rajnov	Biology	127
18	84515	Levie	Biology	126
19	33349	Peck	Biology	125

Pembahasan dan Analisis

Kode implementasi digunakan untuk mengambil data mahasiswa ber-SKS di atas 120 dan menyajikannya secara terurut berlapis (berdasarkan prodi, lalu SKS, lalu nama). Data diambil dari tabel student. Baris ke-1 digunakan untuk memproyeksikan seluruh kolom. Baris ke-2 menunjukkan akses ke tabel student. Baris ke-3 menunjukkan filter nilai tot_cred di atas 120. Baris ke-4 menunjukkan instruksi ORDER BY komposit, di mana data diurutkan berjenjang: prioritas pertama adalah program studi dari A-Z (ASC), prioritas kedua jika ada kesamaan prodi diurutkan berdasarkan SKS dari terbesar ke terkecil (DESC), dan prioritas ketiga diurutkan dari nama A-Z (ASC).

Dari percobaan dan analisis yang telah dilakukan, apabila baris 4 tidak ada maka hasilnya data akan tetap terfilter sesuai syarat SKS > 120 namun tampilannya acak/tidak rapi secara alfabetis. Sedangkan apabila pada baris 4 susunan urutan kolomnya ditukar (misal tot_cred DESC di depan), maka hasilnya pengelompokan prodi akan berantakan karena sistem memprioritaskan nilai SKS tertinggi terlebih dahulu terlepas dari apa nama prodinya, sehingga supaya dapat mengambil data yang dikelompokkan secara terurut dan hierarkis harus menempatkan nama kolom pada klausa ORDER BY secara berurutan sesuai prioritas dengan mempertimbangkan penggunaan keyword ASC atau DESC.

6. Kesimpulan

Berdasarkan percobaan Data Query SELECT yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

- **Pencapaian Tujuan:** Tujuan percobaan telah tercapai dengan baik. Praktikan telah berhasil mengimplementasikan operasi SELECT untuk memproyeksikan kolom, menerapkan kriteria seleksi baris secara spesifik menggunakan klausa WHERE beserta operator relasional (>, <=) dan logika (AND, BETWEEN), serta mengurutkan data secara hierarkis menggunakan klausa ORDER BY.
- **Hubungan Hasil Percobaan dan Dasar Teori:** Hasil percobaan sejalan dengan dasar teori operasional basis data relasional. Percobaan membuktikan bahwa pendefinisian nama

kolom pada perintah SELECT secara langsung membatasi data yang diproyeksikan. Pada operasi seleksi, klausa WHERE terbukti secara akurat memfilter baris data. Penggunaan operator AND membuktikan teori irisan (kedua syarat wajib terpenuhi), dan fungsi BETWEEN membuktikan batasan nilai rentang yang bersifat inklusif. Selain itu, konsep hierarki pengurutan terbukti melalui ORDER BY, di mana sistem memprioritaskan penyortiran data (menggunakan ASC atau DESC) murni berdasarkan urutan kolom yang dituliskan pertama kali pada sintaks.

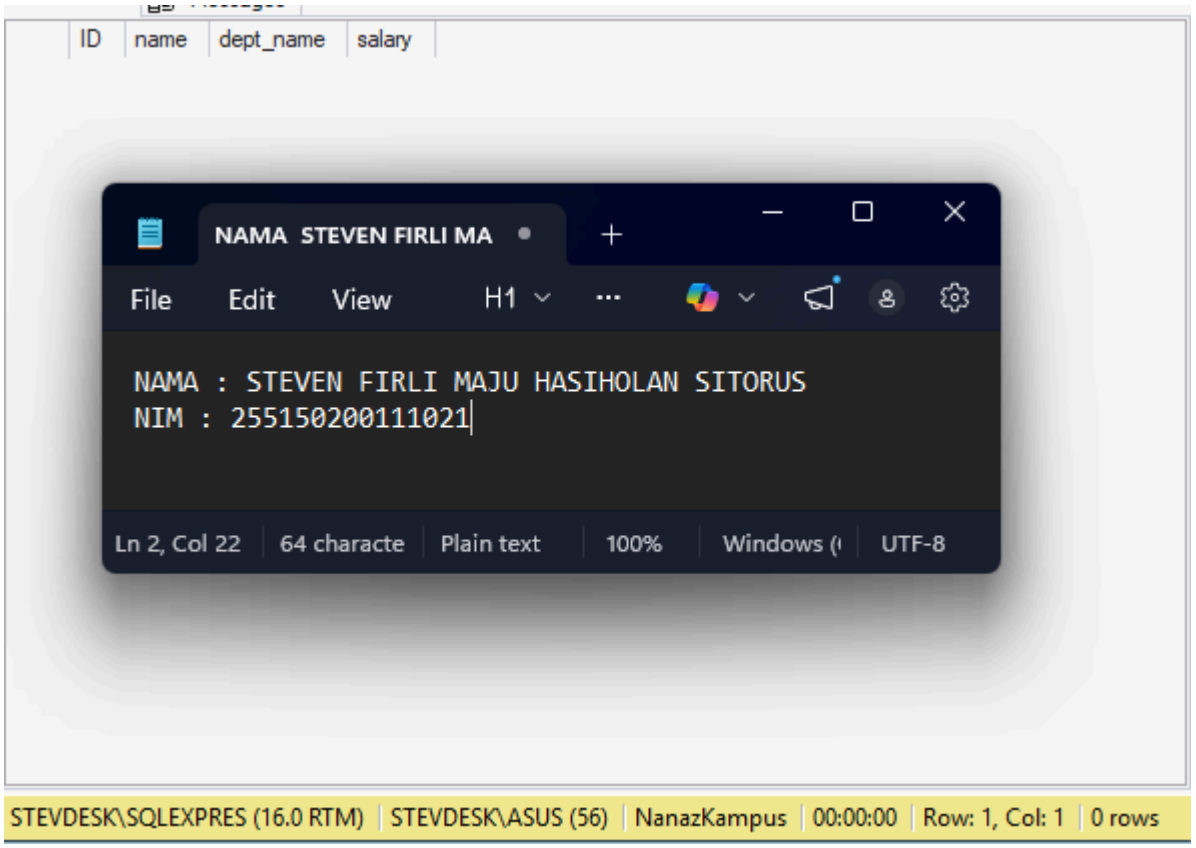
- **Faktor yang Mempengaruhi Hasil dan Potensi Kesalahan:** Faktor utama yang menentukan keberhasilan query adalah presisi penulisan sintaks SQL dan pemahaman logika batasan nilai. Kesalahan yang sangat mungkin terjadi meliputi: keliru mendefinisikan batas awal pada fungsi BETWEEN yang menyebabkan data tidak sesuai kriteria instruksi, terlewatnya penggunaan tanda kutip tunggal ('...') saat menyeleksi data string/varchar (seperti 'Statistics') yang berakibat pada syntax error, serta kesalahan penempatan urutan kolom pada ORDER BY yang dapat mengacaukan prioritas pengelompokan data pada tampilan keluaran akhir.

DATA MANIPULATION LANGUANGE

1. **Implementasi**

```
SELECT * INTO copy_instructor_nim
FROM instructor
WHERE 1 = 0;
```

Tampilan Keluaran



Pembahasan dan Analisis

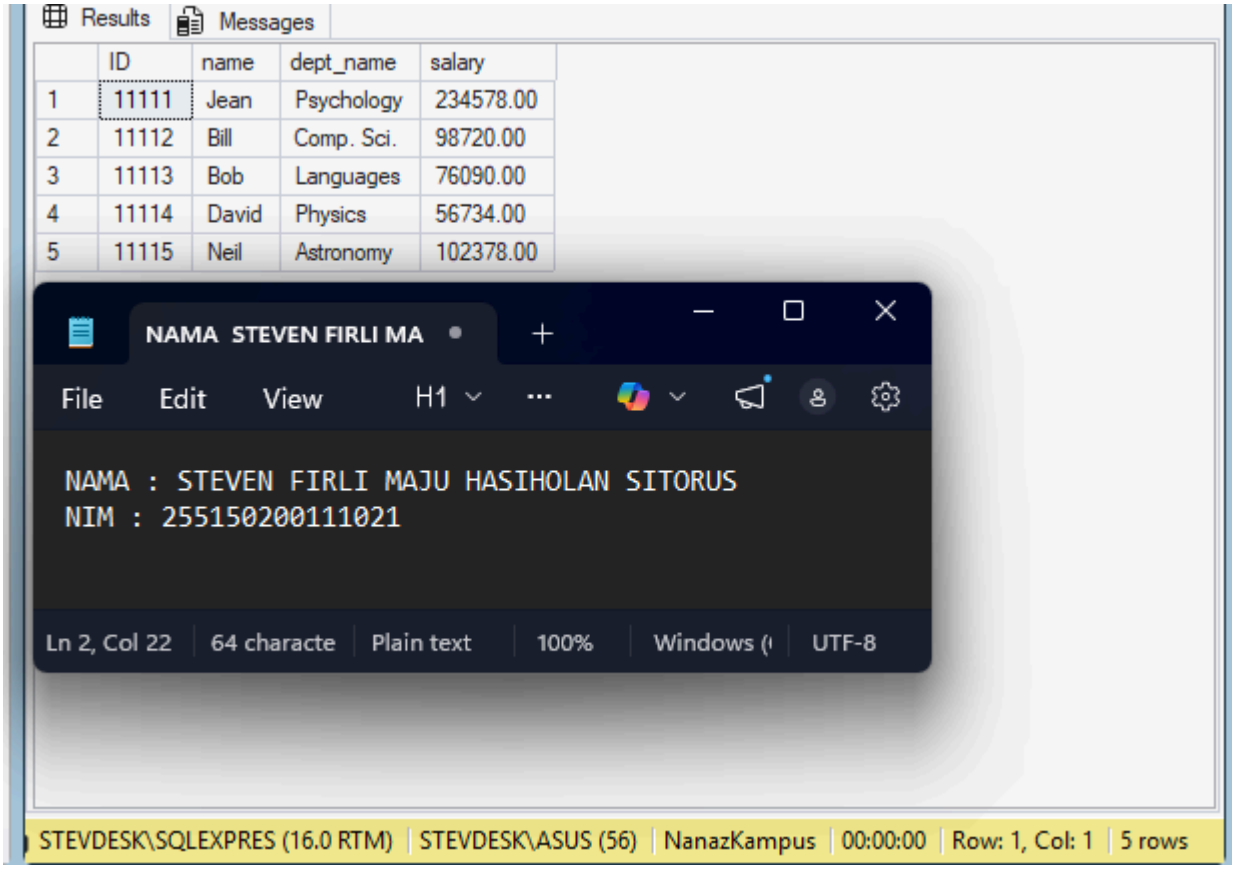
Perintah SELECT * INTO tabel_baru membuat tabel baru sekaligus menyalin struktur dari hasil

query. Klausa WHERE 1 = 0 (kondisi false) memastikan bahwa tidak ada baris data (record) dari tabel asli yang ikut tersalin. Hasilnya, kamu mendapatkan tabel berstruktur identik tapi murni kosong.

2. Implementasi

```
INSERT INTO copy_instructor_nim (ID, name, dept_name, salary)
VALUES
('11111', 'Jean', 'Psychology', 234578),
('11112', 'Bill', 'Comp. Sci.', 98720),
('11113', 'Bob', 'Languages', 76090),
('11114', 'David', 'Physics', 56734),
('11115', 'Neil', 'Astronomy', 102378);
```

Tampilan Keluaran



	ID	name	dept_name	salary
1	11111	Jean	Psychology	234578.00
2	11112	Bill	Comp. Sci.	98720.00
3	11113	Bob	Languages	76090.00
4	11114	David	Physics	56734.00
5	11115	Neil	Astronomy	102378.00

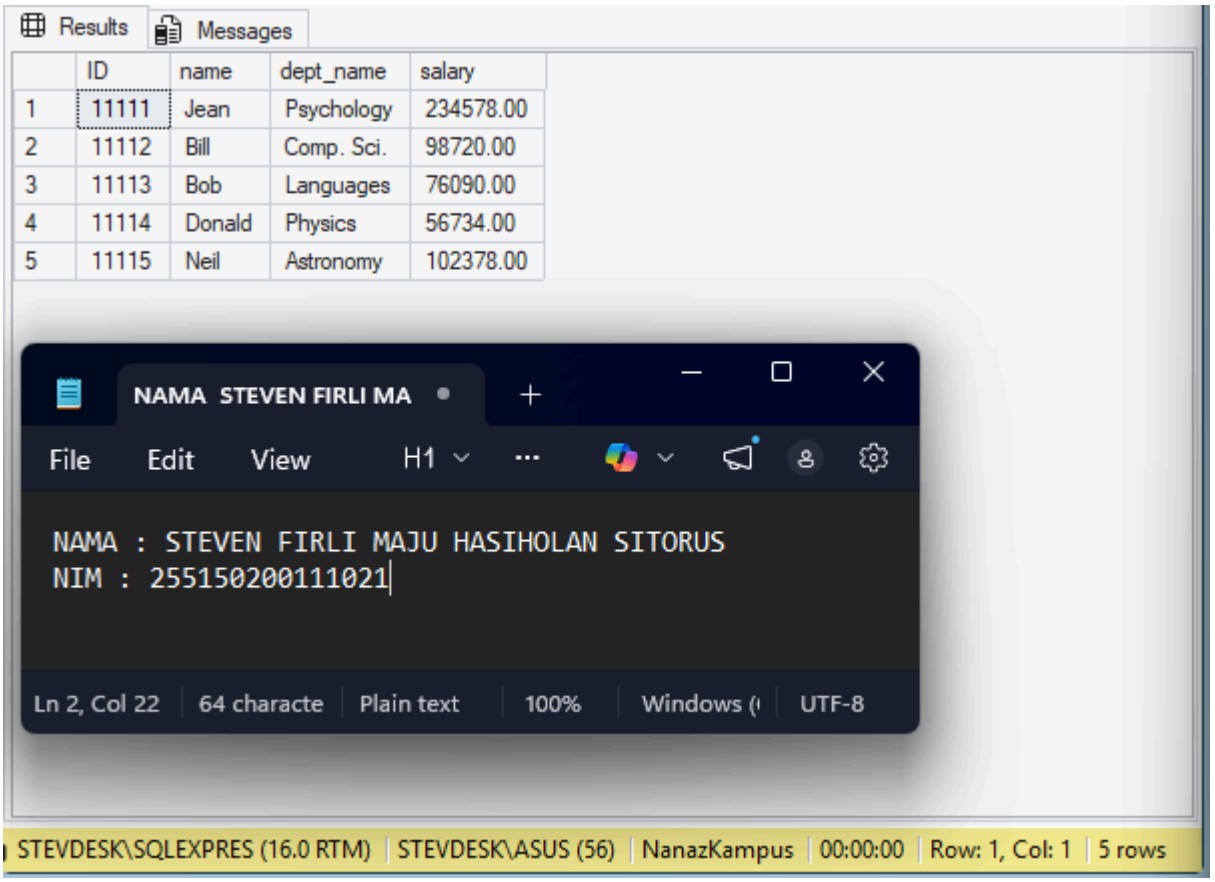
Pembahasan dan Analisis

Perintah DML INSERT INTO digunakan untuk memasukkan record baru. Menyebutkan nama kolom (ID, name, dept_name, salary) setelah nama tabel adalah praktik terbaik (best practice) untuk mencegah error jika urutan tabel diubah. Format angka pada kolom salary ditulis tanpa tanda kutip karena tipe datanya numerik, sementara teks/ID memakai kutip tunggal (').

3. Implementasi

```
UPDATE copy_instructor_nim
SET name = ‘Donald’
WHERE ID = ‘11114’
```

Tampilan Keluaran



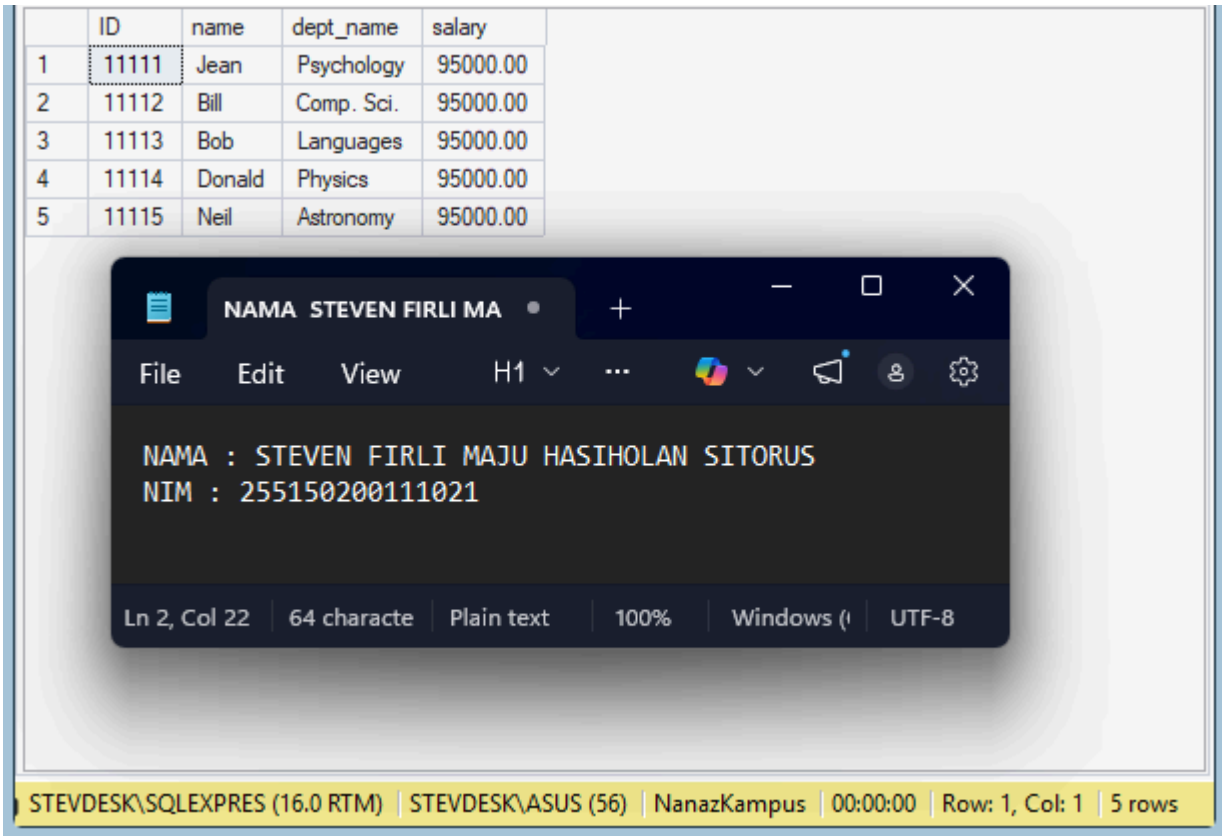
Pembahasan dan Analisis

Perintah UPDATE digunakan untuk memodifikasi data yang sudah ada. SET name = 'Donald' adalah eksekusi pengubahannya. Penggunaan klausa WHERE ID = '11114' sangat krusial; karena ID adalah Primary Key (unik), ini menjamin bahwa hanya baris milik David yang berubah. Jika WHERE lupa ditulis, sistem akan mengubah semua nama di tabel menjadi Donald.

4. Implementasi

```
UPDATE copy_instructor_nim
SET salary = 95000;
```

Tampilan Keluaran



Pembahasan dan Analisis

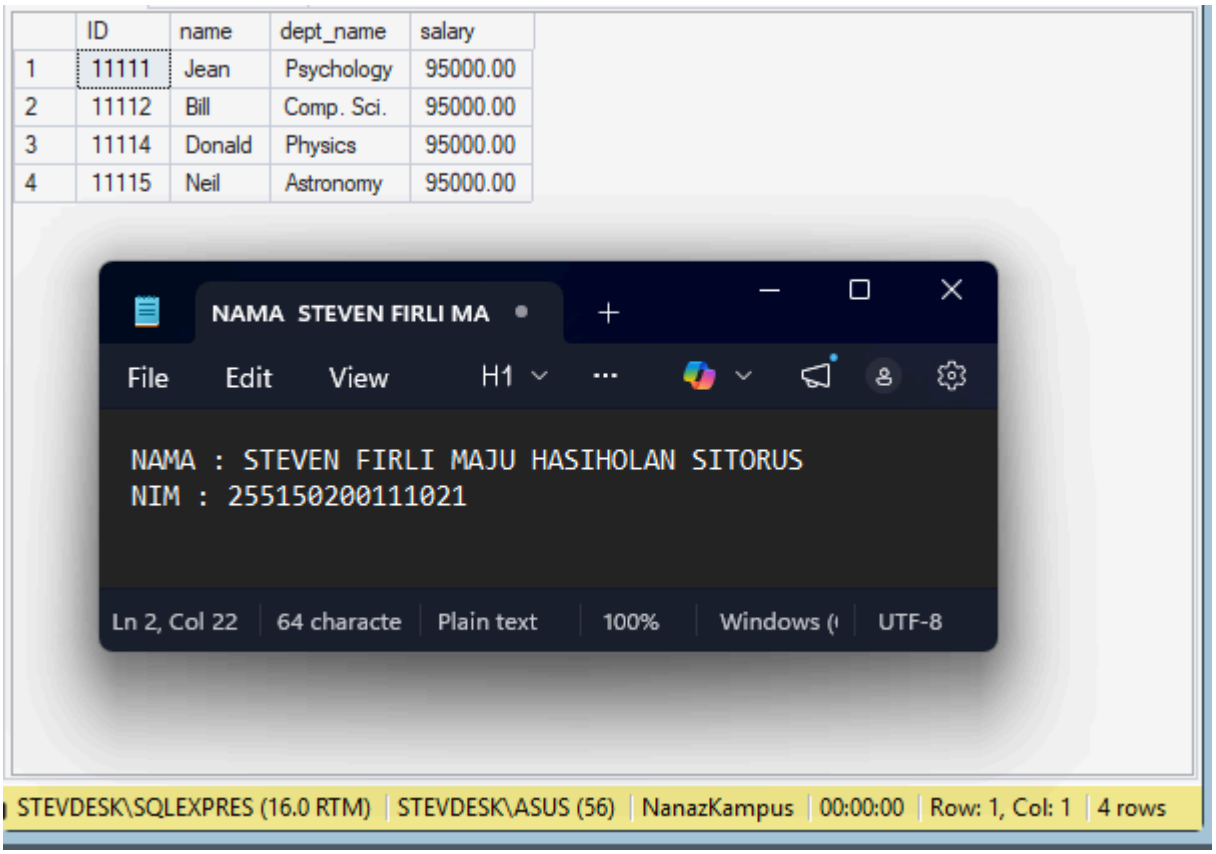
Kode implementasi digunakan untuk memodifikasi data salary secara serentak. Data yang diubah berada pada tabel copy_instructor_nim melalui operasi UPDATE. Baris ke-1 digunakan untuk menentukan tabel target yang akan diubah datanya. Baris ke-2 menunjukkan perintah untuk menetapkan nilai kolom salary menjadi 95000.

Dari percobaan dan analisis yang telah dilakukan, apabila baris 2 tidak ada maka hasilnya akan terjadi error syntax karena perintah UPDATE membutuhkan klausa SET. Karena tidak ada klausa penentu (seperti klausa WHERE pada baris 3), maka hasilnya seluruh baris pada tabel akan terubah gajinya menjadi 95000, sehingga supaya dapat mengambil/mengubah data spesifik saja seharusnya ditambahkan WHERE dengan mempertimbangkan kriteria tertentu (walau pada soal ini memang diminta mengubah semua).

5. Implementasi

```
1  DELETE FROM copy_instructor_nim
2  WHERE name =’Bob’;
```

Tampilan Keluaran



	ID	name	dept_name	salary
1	11111	Jean	Psychology	95000.00
2	11112	Bill	Comp. Sci.	95000.00
3	11114	Donald	Physics	95000.00
4	11115	Neil	Astronomy	95000.00

Pembahasan dan Analisis

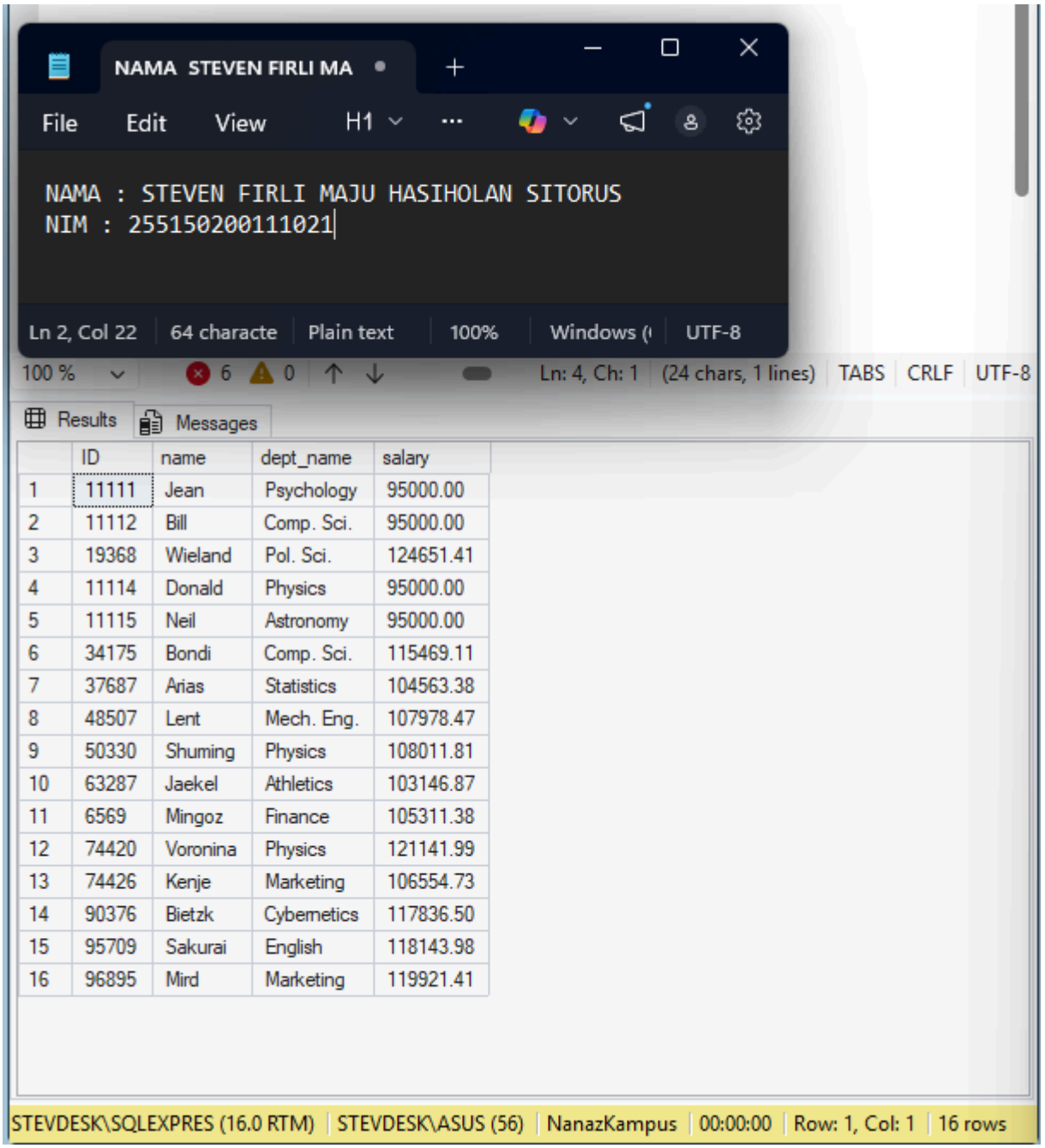
Kode implementasi digunakan untuk menghapus data secara spesifik. Data dihapus dari tabel copy_instructor_nim melalui operasi DELETE. Baris ke-1 digunakan untuk memberikan perintah penghapusan data dari tabel. Baris ke-2 menunjukkan kondisi filter di mana kolom name harus bernilai 'Bob'.

Dari percobaan dan analisis yang telah dilakukan, apabila baris 2 tidak ada maka hasilnya seluruh data di dalam tabel copy_instructor_nim akan terhapus semua secara tidak sengaja. Sehingga supaya dapat mengambil dan menghapus data Bob saja harus menyertakan klausa WHERE dengan mempertimbangkan kondisi nama yang spesifik.

6. **Implementasi**

```
1  INSERT INTO copy_instructor_nim (ID, name, dept_name, salary)
2  SELECT ID, name, dept_name, salary
3  FROM instructor
4  WHERE salary > 100000;
```

Tampilan Keluaran



	ID	name	dept_name	salary
1	11111	Jean	Psychology	95000.00
2	11112	Bill	Comp. Sci.	95000.00
3	19368	Wieland	Pol. Sci.	124651.41
4	11114	Donald	Physics	95000.00
5	11115	Neil	Astronomy	95000.00
6	34175	Bondi	Comp. Sci.	115469.11
7	37687	Arias	Statistics	104563.38
8	48507	Lent	Mech. Eng.	107978.47
9	50330	Shuming	Physics	108011.81
10	63287	Jaekel	Athletics	103146.87
11	6569	Mingoz	Finance	105311.38
12	74420	Voronina	Physics	121141.99
13	74426	Kenje	Marketing	106554.73
14	90376	Bietzk	Cybernetics	117836.50
15	95709	Sakurai	English	118143.98
16	96895	Mird	Marketing	119921.41

Pembahasan dan Analisis

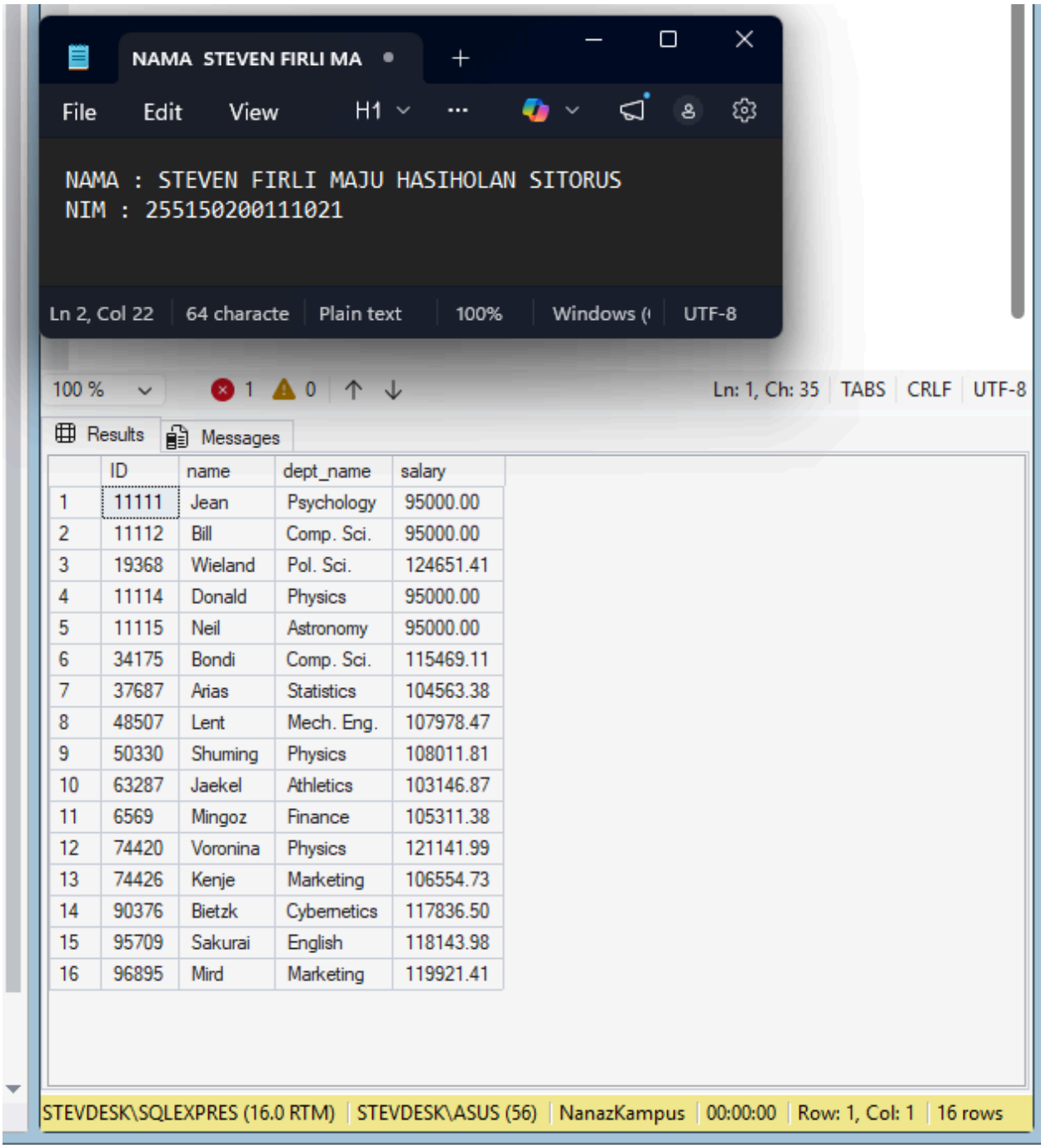
Kode implementasi digunakan untuk menambahkan baris data baru dari sumber tabel lain. Data dimasukkan ke tabel copy_instructor_nim dan diambil dari tabel instructor melalui operasi INSERT dan Subquery. Baris ke-1 digunakan untuk mendefinisikan tabel target beserta kolom-kolomnya. Baris ke-2 menunjukkan perintah SELECT untuk menentukan kolom apa saja yang ditarik. Baris ke-3 menunjukkan asal tabel sumber data. Baris ke-4 menunjukkan kondisi filter gajinya.

Dari percobaan dan analisis yang telah dilakukan, apabila baris 3 tidak ada maka hasilnya sistem akan error karena sumber datanya tidak diketahui. Sedangkan apabila baris 4 tidak ada maka hasilnya seluruh baris data dari tabel instructor akan masuk semua, sehingga supaya dapat mengambil data yang sesuai ketentuan harus menggunakan subquery dengan mempertimbangkan klausa WHERE salary > 100000.

7. **Implementasi**

```
SELECT * FROM copy_instructor_nim;
```

Tampilan Keluaran



	ID	name	dept_name	salary
1	11111	Jean	Psychology	95000.00
2	11112	Bill	Comp. Sci.	95000.00
3	19368	Wieland	Pol. Sci.	124651.41
4	11114	Donald	Physics	95000.00
5	11115	Neil	Astronomy	95000.00
6	34175	Bondi	Comp. Sci.	115469.11
7	37687	Arias	Statistics	104563.38
8	48507	Lent	Mech. Eng.	107978.47
9	50330	Shuming	Physics	108011.81
10	63287	Jaekel	Athletics	103146.87
11	6569	Mingoz	Finance	105311.38
12	74420	Voronina	Physics	121141.99
13	74426	Kenje	Marketing	106554.73
14	90376	Bietzk	Cybernetics	117836.50
15	95709	Sakurai	English	118143.98
16	96895	Mird	Marketing	119921.41

Pembahasan dan Analisis

Kode implementasi digunakan untuk mengambil data secara keseluruhan. Data diambil dari tabel copy_instructor_nim melalui operasi SELECT.

8. Kesimpulan

Berdasarkan percobaan Data Manipulation Language (DML) yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

- **Pencapaian Tujuan:** Tujuan percobaan telah tercapai. Praktikan telah berhasil mengimplementasikan operasi DML secara langsung pada database (seperti INSERT, UPDATE, dan DELETE), serta mampu menggunakan SELECT untuk memverifikasi dan menampilkan perubahan data pada tabel copy_instructor_nim.
- **Hubungan Hasil Percobaan dan Dasar Teori:** Hasil percobaan sejalan dengan dasar teori operasional SQL. Percobaan membuktikan secara langsung peran krusial dari klausa WHERE. Sesuai teori, perintah UPDATE atau DELETE yang dieksekusi tanpa WHERE akan berdampak pada seluruh baris tabel (terbukti saat mengubah seluruh salary menjadi 95000). Sebaliknya, penggunaan WHERE terbukti membatasi manipulasi hanya pada data spesifik (terbukti saat menghapus record bernama 'Bob'). Selain itu, teori integrasi perintah juga terbukti dapat diterapkan melalui penggabungan operasi INSERT dengan metode Subquery (SELECT dari tabel lain).
- **Faktor yang Mempengaruhi Hasil dan Potensi Kesalahan:** Faktor utama yang memengaruhi hasil eksekusi adalah ketepatan penulisan sintaks SQL (seperti penggunaan kutip tunggal untuk string) dan kelengkapan klausa instruksi. Kemungkinan kesalahan atau human error paling fatal dalam materi DML ini adalah kelalaian atau kelupaan praktikan dalam menuliskan klausa WHERE sebagai filter, yang dapat menyebabkan data penting terhapus atau berubah secara masif dan tidak disengaja. Selain itu, ketiadaan klausa wajib seperti SET pada UPDATE atau FROM pada SELECT akan otomatis menyebabkan sistem menghasilkan syntax error.

DAFTAR PUSTAKA

-